

Rozwiązanie: Czy dojadę? – monitor opóźnień pociągów

Michał Król
08.08.2026

Opis ogólny i przeznaczenie aplikacji

- **Przeznaczenie aplikacji:** Dla osób takie jak ja, które chcą łatwo sprawdzić stan opóźnień na wybranych stacjach kolejowych, aby optymalnie zaplanować sposób dojazdu do pracy lub powrotu do domu, w przypadku gdy pojawia się utrudnienia na trasie. Co prawda limit tokenów uniemożliwił szybszą pracę nad jej rozwojem, ale już coś jest. Natomiast na ostatnich slajdach umieściłem planowaną roadmapę dalszego rozwoju aplikacji 😊
- **Opis ogólny działania:** Aplikacja webowa wyświetlająca na bieżąco rozkład odjazdów i przyjazdów pociągów z wybranej stacji wraz z aktualnymi opóźnieniami, pobieranymi z oficjalnego API PKP PLK. Użytkownik wybiera stację i widzi elektroniczną tablicę odjazdów, aktualizowaną automatycznie w tle. W przyszłości planowane jest dodanie powiadomień push, które z wyprzedzeniem poinformują użytkownika o skumulowanym opóźnieniu na wybranych stacjach kolejowych, co ułatwi planowanie

Wymagania funkcjonalne

- Użytkownik może wyszukać i wybrać stację kolejową z listy.
- Aplikacja pobiera i wyświetla aktualną tablicę odjazdów/przyjazdów dla wybranej stacji.
- System automatycznie odświeża dane o opóźnieniach w tle, bez potrzeby ręcznego odświeżania strony.
- Aplikacja respektuje limity zapytań do API PKP PLK (poller z logiką "wake/sleep", by nie przekroczyć limitów).
- Użytkownik widzi status pociągu (na czas / opóźniony, z liczbą minut opóźnienia).

Podpowiedź: Wymagania funkcjonalne to lista konkretnych akcji, zadań i operacji, które system musi wykonywać, aby użytkownik mógł zrealizować swój cel.

Użyta technologia i języki programowania

- **Języki programowania:** TypeScript (cały kod aplikacji i testy)
- **Użyte technologie / frameworki:**
 - Next.js 16 (App Router) — frontend + backend (API routes) w jednym
 - React 19
 - Tailwind CSS 4 — stylowanie (+ next-themes do trybu jasny/ciemny)
 - Zod 4 — walidacja/parsowanie odpowiedzi z API PKP na granicy schematu
 - Vitest + Testing Library (@testing-library/react, user-event, jest-dom) + jsdom — testy jednostkowe/komponentowe
 - ESLint 9 (eslint-config-next, eslint-plugin-testing-library) — lint
 - Zewnętrzne API: PKP PLK (pdp-api.plk-sa.pl) — dane o pociągach (na żywo lub mock z fixture'ów JSON)
 - Hosting/deploy: Railway (pojedyncza replika, stan w pamięci procesu — patrz AGENTS.md)
- **Narzędzie Vibe Coding:** Claude Code

Podpowiedź: Wypisz tutaj konkretne języki, biblioteki i narzędzia, z których faktycznie zbudowana jest Twoja aplikacja.

Podstawowe funkcjonalności - max. 3

- **Funkcjonalność 1:** Wyszukiwanie stacji i wyświetlanie tablicy odjazdów
- **Co się dzieje w aplikacji (reakcja systemu):** Po wybraniu stacji aplikacja pobiera dane z oficjalnego API PKP PLK Open Data i renderuje listę najbliższych odjazdów/przyjazdów wraz ze statusem pociągu.
- **Wartość dla użytkownika:** Użytkownik w kilka sekund sprawdza aktualny rozkład stacji z dowolnego miejsca, zamiast czekać na informację na peronie lub zgadywać, czy pociąg się spóźni.

Podpowiedź: Ta sekcja ma pokazać tzw. ścieżkę użytkownika. Co dokładnie dzieje się pod maską aplikacji w momencie kliknięcia w dany element interfejsu i jaką korzyść (wartość) to przynosi?

Podstawowe funkcjonalności - max. 3

- **Funkcjonalność 2:** Automatyczne odświeżanie opóźnień w tle (poller z logiką wake/sleep)
- **Co się dzieje w aplikacji (reakcja systemu):** Backend cyklicznie odpytuje API PKP PLK w tle (POLL_INTERVAL_MS) i aktualizuje dane o opóźnieniach, jednocześnie pilnując limitów zapytań — "usypia" odpytywanie, gdy nikt aktywnie nie korzysta z danej stacji.
- **Wartość dla użytkownika:** Użytkownik zawsze widzi świeże dane bez ręcznego odświeżania strony, a aplikacja nie przekracza limitów darmowego API, więc działa stabilnie.

Podpowiedź: Ta sekcja ma pokazać tzw. ścieżkę użytkownika. Co dokładnie dzieje się pod maską aplikacji w momencie kliknięcia w dany element interfejsu i jaką korzyść (wartość) to przynosi?

Podstawowe funkcjonalności - max. 3

- **Funkcjonalność 3:** Wskaźnik statusu stacji i szczegóły połączenia
 - **Co się dzieje w aplikacji (reakcja systemu):** Aplikacja wizualnie oznacza status każdego połączenia (np. na czas / opóźnione) na liście odjazdów, a po wybraniu konkretnego pociągu użytkownik może zobaczyć szczegóły danego połączenia.
 - **Wartość dla użytkownika:** Użytkownik od razu, na pierwszy rzut oka, widzi, które pociągi są opóźnione, a jeśli potrzebuje więcej informacji o konkretnym kursie, np. z którego peronu czy toru odjeżdża albo jaki jest numer pociągu lub przewoźnik — ma je w zasięgu jednego kliknięcia, bez przeszukiwania osobnych rozkładów.

Podpowiedź: Ta sekcja ma pokazać tzw. ścieżkę użytkownika. Co dokładnie dzieje się pod maską aplikacji w momencie kliknięcia w dany element interfejsu i jaką korzyść (wartość) to przynosi?

Podstawowe zabezpieczenia

- **Ochrona sekretów (Zmienne środowiskowe):** Klucz dostępowy do API PKP PLK (PKP_API_KEY) jest przechowywany jako zmienna środowiskowa na Railway, niewidoczna w kodzie ani w przeglądarce użytkownika. Podobnie jak i pozostałe zmienne/
- **Dane w zabezpieczonym backendzie:** Aplikacja nie przechowuje danych użytkownika ani żadnych trwałych danych osobowych — dane o odjazdach są jedynie tymczasowo cache'owane w pamięci serwera (z ograniczonym czasem życia).
- **Szyfrowanie połączenia:** Komunikacja odbywa się przez szyfrowany protokół HTTPS, zapewniany automatycznie przez domenę Railway (*.up.railway.app).

Podpowiedź: Tutaj pokazujesz, że aplikacja jest bezpieczna i stabilna od strony technicznej.

Link do rozwiązania

- **Link do rozwiązania:**

- GitHub: https://github.com/michal-krol/delay_monitor
- Railway: <https://delay-monitor-web-production.up.railway.app/>
- Roadmapa z pomysłami co dalej 😊: https://github.com/michal-krol/delay_monitor/blob/main/roadmapa-czy-dojade.jpeg

- **Kluczowe wnioski z indywidualnego wdrożenia:**

- Praca z zewnętrznym, publicznym API (PKP PLK Open Data) wymusiła projektowanie aplikacji od razu pod kątem limitów zapytań — mechanizm odpytывania w tle z logiką wake/sleep oraz priorytetyzacja stacji według zainteresowania użytkowników okazały się kluczowym elementem architektury, a nie dodatkiem dorzuconym na końcu. Podejście spec-first (najpierw pełna specyfikacja, potem implementacja z Claude Code) pozwoliło szybciej iterować nad architekturą
- Sprawdziłem, czy da się pokazywać użytkownikowi konkretną przyczynę opóźnień pociągu (np. „awaria”, „roboty na trasie”). Technicznie dało się to zrobić — ale po analizie uznaliśmy, że pokrycie takich informacji byłoby częściowe (drobne, codzienne opóźnienia najpewniej nie mają tam żadnego opisu), a koszt pozyskania tych danych zbyt wysoki względem realnej korzyści. Decyzję i uzasadnienie spisaliśmy, żeby nie trzeba było tej analizy powtarzać od zera w przyszłości. Wniosek ogólny: nie każdy zbadany pomysł trzeba wdrożyć — czasem najbardziej wartościowym efektem analizy jest świadome „nie teraz”, z jasnym uzasadnieniem dlaczego.
- Błędy związane ze strefami czasowymi czasem ujawniają się dopiero na produkcji -> trzeba zwracać uwagę na różnice w interpretacji czasu w systemach i api. System czasem zwraca godzinę bez informacji, w jakiej strefie czasowej ona jest (np. bez dopisku, że to czas polski). Mój komputer w Polsce interpretował to poprawnie „przez przypadek” — bo sam też jest ustawiony na polską strefę czasową. Serwer, na którym aplikacja działa produkcyjnie, ma jednak inną strefę domyślną — i ten sam kod przesuwał każdy pociąg o dwie godziny. Pociąg, który już odjechał, wyglądał jak dopiero nadchodzący. Wniosek ogólny: to, że coś działa poprawnie na komputerze programisty, nie mówi o tym, czy zadziała na serwerze produkcyjnym — trzeba to jawnie sprawdzić w warunkach zbliżonych do tych na produkcji, nie polegać na przypadkowej zgodności.
- Więcej danych nie zawsze znaczy lepiej — trzeba policzyć koszt. Jedna z opcji API pozwalała pobierać znacznie więcej szczegółów o każdym pociągu (całą jego trasę, a nie tylko fragment potrzebny akurat na danej stacji). Włączenie tej opcji zwiększało ilość pobieranych danych ponad dwunastokrotnie — z 680 KB do 8,6 MB, przy każdym odświeżeniu co półtorej minuty — a przecież z tych dodatkowych danych aplikacja wcale nie korzystała. To był bezpośredni powód sporadycznych błędów odświeżania na produkcji. Po stworzeniu rozwiązania warto myśleć nad dalszą optymalizacją lub refaktorem.

Elementy graficzne do wykorzystania

Boxy do wyróżnienia najważniejszych informacji

